

Orator

Sec

Neural speech-analysis SaaS - Next.js 14 (App Router) + Supabase

Date: 2026-06-22

Branch: claude/compassionate-brahmagupta-68pqyf

Scope: full codebase, all API routes, RLS schemas, and the live Supabase project

Executive summary

A full security pass was run across the application code and the connected Supabase project. One critical data-exposure flaw was found and fixed in the database, several high-severity code vulnerabilities were remediated, and the Supabase advisor warnings were cleared (security and performance). All changes type-check cleanly.

Headline result:

- CRITICAL fixed: any signed-in user could download every other user's private speech recordings (over-permissive storage RLS policy). Closed in the live database.
- 4 critical/high code issues fixed: command injection (RCE), two SSRF vectors, and an unauthenticated paid-LLM endpoint.
- 9 hardening fixes: admin-token forgery, email HTML injection, timing-safe secret checks, upload size/type limits, and removal of permissive insert policies.
- Supabase advisors: security warnings reduced from 8 to 2 (the 2 remaining are by-design or an auth-console toggle); 35+ performance warnings cleared.
- Dependencies: high-severity npm advisories reduced from 14 to 8 (non-breaking fixes applied; remainder require a Next.js major upgrade - see recommendations).

Part A - Supabase (applied to the live database)

All migrations below were applied to project tmjyujpprveeqhorjcg and mirrored into the repo SQL files. They are idempotent and recorded in supabase/security_hardening.sql.

A1. CRITICAL - private speech recordings were world-readable

CRITICAL - FIXED

The storage policy "Service reads speeches" granted SELECT on every object in the private 'speeches' bucket to the public role (anon + authenticated). Combined with the public anon key, any authenticated user could list and download any other user's uploaded speech/video files - a horizontal access-control break (IDOR). The server's service-role key already bypasses RLS, so the policy was never needed.

Fix: dropped the policy. Per-user "Users read own speeches" remains, so owners still read their own files and the server still reads everything via the service role.

A2. brand-assets bucket allowed full file enumeration

LOW - FIXED

The public 'brand-assets' bucket had a broad SELECT policy that let any client LIST every file. Public buckets serve objects via their public URL without such a policy, so it was dropped (image src references are unaffected).

A3. Permissive INSERT policies on feedback_points / engagement_timeline

MEDIUM - FIXED

Both tables had INSERT policies using WITH CHECK (true), granting insert to every role with no ownership tie - a user could inject fake feedback/timeline rows against any analysis. The only legitimate writer is the Modal GPU callback (service-role key, bypasses RLS), so the policies were dropped.

A4. SECURITY DEFINER function hardening

WARN - FIXED

public.handle_new_user() had a mutable search_path and was directly executable by anon / authenticated via PostgREST RPC. Fix: pinned search_path = " and revoked EXECUTE from public/anon/authenticated. The signup trigger still fires normally.

A5. RLS performance - auth.uid() re-evaluated per row

PERF - FIXED

35+ policies called auth.uid() once per row. Rewrote them as (select auth.uid()) so Postgres evaluates it once per query. Access logic is identical; clears advisor 0003.

A6. Missing covering indexes on foreign keys

PERF - FIXED

Added 11 covering indexes for foreign keys flagged by advisor 0001 (analyses.user_id, feedback_points.analysis_id, all clipflow_* and agent_* FKs, etc.).

A7. Remaining advisor items (not auto-fixable / by design)

- access_requests INSERT WITH CHECK (true): intentional - the public request-access form must accept anonymous submissions. Recommend app-layer rate limiting (see C-list).
- Leaked password protection disabled: enable in Supabase Auth -> Settings (console toggle, cannot be set via SQL).

Part B - Application code fixes

B1. Command injection / RCE in ClipFlow renderer

CRITICAL - FIXED

lib/clipflow/clipper.ts built shell command strings with a YouTube video ID interpolated directly into yt-dlp/ffmpeg via exec(). The ID came from a user-pasted URL with no character validation, so a crafted URL could execute arbitrary commands as the server user.

Fix: (1) switched all three exec() calls to execFile() with argv arrays (no shell); (2) strict validation - video IDs must match `^[A-Za-z0-9_-]{11}$` at parse time (lib/clipflow/youtube.ts) and again before rendering. Channel IDs/handles/usernames are now allow-listed too.

B2. SSRF via user-supplied URLs (calendar ICS + website brand import)

CRITICAL/HIGH - FIXED

Both the connected-calendar ICS fetch (lib/gigs/ics.ts) and the brand/website extractor (lib/brand/extract.ts) fetched arbitrary user URLs with follow-redirects and no protection against internal targets - allowing requests to localhost, RFC1918 ranges, and the cloud metadata endpoint (169.254.169.254). The brand extractor also reflected scraped content back to the caller, enabling exfiltration of internal responses.

Fix: added a shared guard lib/security/ssrf.ts (safeFetch / assertPublicUrl) that resolves the hostname, rejects loopback/private/link-local/CGNAT/multicast addresses (IPv4 + IPv6, incl. IPv4-mapped), and follows redirects manually - re-validating every hop. Both call sites now route through it.

B3. Unauthenticated, unmetered paid-LLM endpoint

CRITICAL - FIXED

/api/compare-report had no auth check (middleware excludes /api), so anyone on the internet could trigger GPT-4o calls - unbounded paid-API cost. User input was also interpolated into the prompt unvalidated, and missing fields threw unhandled 500s.

Fix: added a getUser() 401 gate; tone/audience/length/sections validated against fixed allow-lists; labels and metrics coerced/bounded; body parsing and the OpenAI call wrapped in try/catch with generic errors.

B4. Forgeable admin approve/deny tokens

HIGH - FIXED

lib/adminToken.ts fell back to a hardcoded HMAC secret ('fallback-secret-change-me') when ADMIN_ACTION_SECRET was unset. Those tokens are the ONLY auth on /api/admin/action (which approves access and mints invite links), so a shipped default allowed self-approval. Fix: throw if the secret is unset (no default) and compare signatures with timingSafeEqual.

B5. HTML injection into admin notification email

HIGH - FIXED

Public, untrusted name/email/reason from the request-access form were interpolated unescaped into the admin notification email (a phishing vector against the admin who holds approve power). Fix: all user values are HTML-escaped in lib/email.ts.

B6. Upload size / type limits

HIGH - FIXED

/api/transcribe buffered and forwarded arbitrarily large blobs to a paid API; the editor upload route buffered whole files in memory and trusted the client-supplied content type. Fix: 25 MB cap on transcribe (matches Whisper's limit); 200 MB cap + extension allow-list + server-derived content type on editor upload.

B7. Non-constant-time secret comparison (cron worker)

MEDIUM - FIXED

/api/clipflow/jobs/run compared the CRON_SECRET bearer token with ==. Switched to a length-checked timingSafeEqual.

Part C - Dependencies

npm audit reported 14 vulnerabilities. The non-breaking fixes were applied (npm audit fix), clearing the high-severity form-data CRLF injection plus js-yaml, minimatch (ReDoS), and uuid issues - down to 8.

The remaining 8 are all in next@14.2.29 (and its bundled postcss) and the eslint glob/minimatch dev chain; clearing them requires a major upgrade. Note: the critical Next.js middleware auth-bypass (CVE-2025-29927, fixed in 14.2.25) does NOT apply here - this app is already on 14.2.29. The remaining Next advisories are mostly DoS / image-optimizer / cache. Recommend upgrading to the latest 14.2.x patch and testing, then evaluating Next 15.

Part D - Recommended follow-ups (not changed in this pass)

These need product decisions or testing and were intentionally left for review:

- Rate limiting on public/abuse-prone routes (request-access, public/inquiry, transcribe, compare-report). The app has none today; add Upstash or similar.
- Agent (Gmail) 'act_directly' autonomy enforces the 'confirm before irreversible' rule only in the system prompt - a malicious email could override it. Add a code-level confirmation gate and validate/allow-list recipients before send.
- CLIPFLOW_TOKEN_SECRET falls back to the service-role key for encrypting OAuth tokens. Require a dedicated secret (no rows are encrypted yet, so it is safe to switch now).
- Timeline clipPath signing (/api/editor/timeline/[id]) and signed-upload routes should verify the referenced path/project belongs to the caller (defense in depth; today they are protected only by the user-id storage namespace).
- Stop returning raw upstream/exec error strings to clients (info disclosure); log server-side and return generic messages.
- Neutralize CSV formula injection in the analysis export (prefix cells starting with = + - @).
- Enable Supabase leaked-password protection; set ADMIN_ACTION_SECRET in all environments (the app now refuses to mint admin links without it).

Files changed: supabase/{schema,brand,gigs,bookings,agent,clipflow,security_hardening}.sql, lib/security/ssrf.ts (new), lib/clipflow/{clipper,youtube}.ts, lib/{adminToken,email}.ts, lib/gigs/ics.ts, lib/brand/extract.ts, app/api/compare-report/route.ts, app/api/clipflow/jobs/run/route.ts, app/api/transcribe/route.ts, app/api/editor/[id]/upload/route.ts, package-lock.json. Verified: tsc --noEmit passes.